Test Plan Document

DockWatch: Docker Container Health Monitoring System

Version 2.0 — May 2026 — Systems Engineering Team

0 Contents

1 Introduction

1.1 Purpose

This document defines the overall testing strategy for DockWatch v1.0. It specifies the scope, approach, resources, and schedule for all testing activities. The goal is to verify that DockWatch meets all functional requirements (FR-01 through FR-06) and selected non-functional requirements defined in the SRS v1.0.

Version 2.0 extends the original plan with a single, unified **Selenium WebDriver test suite** (`tests/selenium_tests.py`) that covers both White Box (WB-01 to WB-11) and Black Box (BB-01 to BB-21) test cases. All 110 test methods run through a real Chromium-based browser, requiring no additional static-analysis tooling.

1.2 Scope

Testing covers:

- All six functional requirements (FR-01 to FR-06)
- Key non-functional requirements: NFR-01 (performance), NFR-02 (resource usage), NFR-03 (availability), NFR-04 (security)
- Four testing levels: unit, integration, system, and technique-driven (white box + black box)
- The web-based dashboard, REST API, WebSocket endpoint, and Docker Engine integration

Out of scope: testing of the Docker Engine itself, host OS configuration, third-party CI/CD tools, and static analysis (SonarQube has been removed from this project).

1.3 References

- DockWatch SRS v1.0, April 15, 2026
- DockWatch Requirement Validation Document v1.0, May 2026
- IEEE Std 829-2008, *IEEE Standard for Software and System Test Documentation*
- Docker Engine API v1.41 Documentation
- OWASP Top 10 Security Risks
- Selenium WebDriver 4 Documentation

1.4 Definitions

Term	Definition
Unit Test	Tests a single function or module in isolation.
Integration Test	Tests the interaction between two or more components.
System Test	End-to-end test of the complete system against requirements.
White Box Test	Derives test cases from internal code structure (branches, paths, boundary values); run here via Selenium through the browser.
Black Box Test	Derives test cases from observable behaviour with no internal knowledge; verified through Selenium UI interactions.
Pass Criterion	The measurable condition that determines a test has passed.
SUT	System Under Test — the DockWatch application.
WB	White Box test identifier prefix (WB-01 to WB-11).
BB	Black Box test identifier prefix (BB-01 to BB-21).

2 Testing Strategy

2.1 Testing Levels

DockWatch is tested at four levels, applied in sequence:

1. **Unit Testing** — Individual backend functions and frontend components tested in isolation.
2. **Integration Testing** — Interaction between backend, Docker Engine API, REST/WebSocket endpoints, and the database.
3. **System Testing** — Full end-to-end scenarios: container failure detection, recovery, dashboard display, historical data retrieval.
4. **White Box & Black Box Testing** — All implemented as Selenium WebDriver tests in `tests/selenium_tests.py`. White box tests drive the browser to exercise internal logic paths (auth, validation, access control). Black box tests verify only externally observable behaviour.

2.2 Testing Approach

Level	Tool	Responsibility
Unit Testing	Selenium (API via `requests`)	Developers
Integration Testing	Selenium + `requests` against live backend	QA Engineers
System Testing	Selenium WebDriver (Chromium, headless)	QA Engineers
White Box (WB-01–11)	Selenium + direct API calls	Developers
Black Box (BB-01–21)	Selenium WebDriver (browser interactions)	QA Engineers
Performance Testing	Locust	QA Engineers
Security Testing	OWASP ZAP + manual review	Security Lead

2.3 Entry and Exit Criteria

Entry Criteria (testing may begin when):

- SRS and Requirement Validation documents are approved.
- Development of the feature under test is complete.
- Test environment (Docker host, database) is configured and accessible.
- Frontend running on `http://localhost:3001`; backend on `http://localhost:8000`.

Exit Criteria (testing phase is complete when):

- All 110 Selenium test methods execute without unhandled exceptions.
- No open critical or high-severity defects remain.
- All high-priority test cases pass.
- All system test scenarios pass their defined acceptance criteria.

3 Test Environment

3.1 Hardware

- Linux-based host (x86_64), minimum 2 vCPUs, 4 GB RAM
- Docker Engine installed and running (API v1.41+)

3.2 Software

- DockWatch backend: Python FastAPI (port 8000)
- DockWatch frontend: React (port 3001)
- Database: SQLite (via SQLAlchemy + Alembic migrations)
- Test containers: at least 5 lightweight Docker containers (e.g., nginx, alpine)
- Browser: Brave/Chrome (latest) — headless mode for Selenium
- Python packages: `selenium`, `requests`, `pytest`

3.3 Network

- Backend API: `http://localhost:8000`
- Frontend: `http://localhost:3001`
- WebSocket: `ws://localhost:8000/ws`

3.4 Running the Test Suite

```
# Install dependencies
pip install selenium requests pytest

# Start DockWatch (from project root)
./start.sh

# Run all 110 Selenium tests
python -m pytest tests/selenium_tests.py -v

# Run only White Box tests
python -m pytest tests/selenium_tests.py -v -k "WB"

# Run only Black Box tests
python -m pytest tests/selenium_tests.py -v -k "BB"
```

4 Unit Test Cases

TC-ID	Test Case	Expected Result	Req.	Priority
UT-01	CPU usage calculation from mock Docker stats	Correct percentage to 2 decimal places	FR-01	**High**
UT-02	RAM usage calculation from mock stats	Correct MB value returned	FR-01	**High**
UT-03	Container status parser for running, exited, unhealthy	Correct state string returned	FR-02	**High**
UT-04	YAML config loader parses valid config	Config object populated correctly	FR-06	**High**
UT-05	YAML config loader on malformed YAML	Exception raised; no crash	FR-06	**Medium**
UT-06	JWT validation accepts valid token	Decoded user payload returned	NFR-04	**High**
UT-07	JWT validation rejects expired token	401 Unauthorized returned	NFR-04	**High**
UT-08	Recovery rule evaluator triggers restart	Returns action type “restart”	FR-03	**High**
UT-09	Recovery rule evaluator triggers notify	Returns action type “notify”	FR-03	**Medium**
UT-10	Telemetry serializer outputs valid JSON	Valid JSON with all required fields	FR-06	**Medium**

Table 1: Unit Test Cases

5 Integration Test Cases

TC-ID	Test Case	Expected Result	Req.	Priority
IT-01	Backend polls live Docker Engine API	Container list returned with PID, CPU, RAM	FR-01	**High**
IT-02	REST API GET `/containers`	Status 200; JSON container list	FR-04, FR-06	**High**
IT-03	REST API GET `/containers/{id}/metrics`	Timestamped metric records returned	FR-04	**High**
IT-04	WebSocket connection; server pushes update	Client receives update within 5 seconds	FR-05	**High**
IT-05	Simulated container crash detected by backend	Failure event logged within 30 seconds	FR-02	**High**
IT-06	Detected failure triggers recovery via Docker SDK	SDK call invoked and action logged	FR-03	**High**
IT-07	Metrics persisted to database after polling	DB record created with correct values	FR-04	**Medium**
IT-08	Frontend re-renders on WebSocket update	DOM updates without page reload	FR-05	**Medium**
IT-09	YAML config change applied without restart	New polling interval takes effect	FR-06	**Medium**

Table 2: Integration Test Cases

6 System Test Cases

6.1 Functional System Tests

TC-ID	Test Case	Expected Result	Req.	Priority
ST-01	Admin opens dashboard; all containers shown with live metrics	All containers visible; metrics update in real time	FR-01, FR-05	**High**
ST-02	Admin stops a container; alert appears on dashboard within 30s	Alert visible within 30 seconds	FR-02, FR-05	**High**
ST-03	System auto-restarts failed container per recovery rule	Container returns to running; event logged	FR-03	**High**
ST-04	QA queries REST API for 7-day historical CPU data	JSON array of timestamped CPU values returned	FR-04	**High**
ST-05	Admin updates YAML alert threshold	New threshold applied in next polling cycle	FR-06	**Medium**
ST-06	10 containers monitored simultaneously	All 10 shown with accurate metrics	FR-01, NFR-05	**Medium**

Table 3: Functional System Test Cases

6.2 Performance Tests

TC-ID	Test Case	Pass Criterion	Req.	Priority
PT-01	100 concurrent requests to GET `/containers` via Locust	95th percentile response $\leq$ 200ms	NFR-01	**High**
PT-02	Monitor 50 containers for 10 minutes	Host CPU overhead $< 5\%$	NFR-02	**High**
PT-03	DockWatch running continuously for 24 hours	No crash; memory stable	NFR-03	**Medium**

Table 4: Performance Test Cases

6.3 Security Tests

TC-ID	Test Case	Pass Criterion	Req.	Priority
SEC-01	Access API endpoint without JWT token	401 Unauthorized returned	NFR-04	**High**
SEC-02	Submit expired JWT token	401 Unauthorized returned	NFR-04	**High**
SEC-03	OWASP ZAP scan against REST API	No high-severity vulnerabilities	NFR-04	**High**
SEC-04	Verify all traffic uses TLS	No plaintext HTTP connections accepted	NFR-04	**High**

TC-ID	Test Case	Pass Criterion	Req.	Priority

Table 5: Security Test Cases

7 White Box Test Cases — Selenium (WB-01 to WB-11)

Approach: White box scenarios are exercised through the running application via Selenium WebDriver and direct REST API calls using the `requests` library. The same correctness properties verified by traditional unit tests (bcrypt hash comparison, JWT validation, input rejection, access control enforcement) are confirmed by observing the application's response to targeted inputs.

Tool: Selenium WebDriver 4 (Chromium headless) + Python `requests`
File: `tests/selenium_tests.py` — classes `TestWB01_*` through `TestWB11_*`
Run: `python -m pytest tests/selenium_tests.py -v -k "WB"`

TC-ID	Feature	Test Cases (method count)	Pass Criterion	Pri.
WB-01	**Password auth** (5 TCs)	TC01 correct password ⇒ login succeeds. TC02 wrong password ⇒ stays on /login. TC03 wrong username ⇒ login fails. TC04 password not in page HTML. TC05 6 failed attempts ⇒ account not accessible.	Each condition produces the expected HTTP status / URL / page state.	**High**
WB-02	**JWT session** (6 TCs)	TC01 protected route without login ⇒ login form shown. TC02 JWT persists across 4 navigations. TC03 logout revokes session. TC04 API returns `access_token` on valid login. TC05 tampered JWT rejected (401/403). TC06 no token rejected (401/403).	JWT lifecycle enforced at every step.	**High**
WB-03	**Container actions** (6 TCs)	TC01 /containers page loads. TC02 container detail page opens. TC03 action buttons (start/stop/restart) present. TC04 API container list returns JSON array. TC05 API stop non-existent ⇒ 400/404. TC06 API restart non-existent ⇒ 400/404.	Actions present; non-existent containers return error codes.	**High**

TC-ID	Feature	Test Cases (method count)	Pass Criterion	Pri.
WB-04	**Container not found** (3 TCs)	TC01 /container/{fake-id} shows error state or redirects. TC02 API start fake ID ⇒ 400/404. TC03 API stop fake ID ⇒ 400/404.	App does not crash; returns appropriate error codes.	**High**
WB-05	**Health check** (5 TCs)	TC01 GET /api/health ⇒ 200. TC02 response has `status` key. TC03 response has `components.database` and `components.docker`. TC04 /api/health in browser shows JSON. TC05 dashboard loads with content after login.	Health schema correct; endpoint always reachable.	**Medium**
WB-06	**Input validation** (8 TCs)	TC01–03 empty username / password / both fields ⇒ login blocked. TC04 short container ID ⇒ 400/404/422. TC05 missing password field ⇒ 400/422. TC06 missing username field ⇒ 400/422. TC07 alert-rules page has form inputs. TC08 settings page has form inputs.	Invalid or missing inputs are rejected; forms present in UI.	**High**
WB-07	**Alert thresholds** (7 TCs)	TC01 /alert-rules page loads. TC02 “Alert” visible on /alerts. TC03 API create rule with 80% threshold. TC04 API create rule with 0% (lower boundary). TC05 API create rule with 100% (upper boundary). TC06 GET channels without token ⇒ 401/403. TC07 GET channels with token ⇒ 200 list.	Boundary thresholds accepted; access control enforced.	**Medium**

TC-ID	Feature	Test Cases (method count)	Pass Criterion	Pri.
WB-08	**Backup** (5 TCs)	TC01 /backup page loads. TC02 GET /api/backup/list without token ⇒ 401/403. TC03 GET /api/backup/list with token ⇒ 200, backups key. TC04 POST /api/backup/create without token ⇒ 401/403. TC05 Backup page has create/backup button.	Auth enforced; backup list and creation accessible to admin.	Medium
WB-09	**Audit log** (6 TCs)	TC01 /audit page loads. TC02 GET /api/audit/logs without token ⇒ 401/403. TC03 with admin token ⇒ 200, `logs` key. TC04 limit=5 returns ≤ 5 entries. TC05 limit=200 rejected (422). TC06 audit page body has audit-related text.	Pagination and access control enforced; log schema correct.	High
WB-10	**Access control** (5 TCs)	TC01 all three container action endpoints blocked without token. TC02 admin accesses /users, /audit, /security, /backup without redirect. TC03 /users accessible to admin. TC04 /security accessible to admin. TC05 GET /api/users without token ⇒ 401/403/404.	Unauthenticated requests blocked; admin role grants full access.	High
WB-11	**Schema validation** (6 TCs)	TC01 empty body on login ⇒ 400/422. TC02 integer credentials ⇒ 400/401/422. TC03 empty body on change-password ⇒ 400/422. TC04 short new password ⇒ 400. TC05 settings page has ≥ 1 form input. TC06 health response has all required schema keys.	Malformed payloads rejected; UI reflects correct schema.	Medium

TC-ID	Feature	Test Cases (method count)	Pass Criterion	Pri.

Table 6: White Box Test Cases — Selenium (WB-01 to WB-11)

WB Summary: 11 test groups, 32 test classes, **62 test methods**.

8 Black Box Test Cases — Selenium (BB-01 to BB-21)

Approach: Black box tests derive cases purely from observable user-facing behaviour. No knowledge of implementation is used. Selenium drives a headless Chromium browser, interacting with the running DockWatch frontend exactly as a user would.

Tool: Selenium WebDriver 4 (Chromium headless)
File: `tests/selenium_tests.py` — classes `TestBB01_*` through `TestBB21_*`
Run: `python -m pytest tests/selenium_tests.py -v -k "BB"`

TC-ID	Feature	Test Cases (method count)	Pass Criterion	Pri.
BB-01	**Login** (3 TCs)	TC01 root URL opens app. TC02 valid credentials redirect off /login. TC03 wrong password stays on /login.	URL and redirect state match expectation.	**High**
BB-02	**Dashboard content** (5 TCs)	TC01 dashboard reachable after login. TC02 “Total Containers” label visible. TC03 “Running” section visible. TC04 “Avg CPU” metric visible. TC05 Logout returns to root.	All KPI labels and logout work as expected.	**High**
BB-03	**Navigation** (6 TCs)	Navigate to /hosts, /alerts, /settings, /alert-rules, /schedules, /stacks.	Each route URL appears in address bar.	**High**
BB-04	**Container actions** (2 TCs)	TC01 click first container link ⇒ detail page. TC02 type in search input ⇒ no crash.	URL contains /container/; search completes silently.	**High**
BB-05	**User management** (1 TC)	Navigate to /users as admin.	URL contains /users; no redirect to login.	**Medium**
BB-06	**Security & audit** (2 TCs)	Navigate to /security and /audit.	Both routes load; URL correct; no redirect.	**High**
BB-07	**Responsive design** (3 TCs)	Load /login at 375×667, 768×1024, and 1920×1080.	Page reachable and not blank at all three sizes.	**Medium**
BB-08	**Error handling** (2 TCs)	TC01 navigate to unknown route. TC02 navigate away and back to /login.	App stays up; login page still reachable after bad route.	**Medium**
BB-09	**Theme switching** (2 TCs)	TC01 click theme toggle. TC02 toggle then reload.	No JS crash; page still loads after reload.	**Low**
BB-10	**Docker resources** (2 TCs)	Navigate to /docker; inspect page text.	URL correct; “Images” keyword visible.	**Medium**

TC-ID	Feature	Test Cases (method count)	Pass Criterion	Pri.
BB-11	**Alerts config** (2 TCs)	"Alert" on /alerts; Create button on /alert-rules.	Keyword present; button found.	Medium
BB-12	**Settings page** (2 TCs)	Section labels visible; submit button present.	≥ 1 of General/Security/API/About visible; button found.	Medium
BB-13	**Filters & sorting** (2 TCs)	Filter dropdown and sort control on /containers.	Controls present (skipped gracefully if absent).	Low
BB-14	**Pagination** (2 TCs)	"Showing"/"Page" text and next button on /containers.	Indicators present when multiple pages exist.	Low
BB-15	**Notifications** (1 TC)	Navigate to /notifications.	URL contains /notifications.	Medium
BB-16	**3D Topology** (1 TC)	Navigate to /topology.	URL contains /topology; no crash.	Medium
BB-17	**Container compare** (2 TCs)	Navigate to /compare; inspect for select elements.	URL correct; selects present if containers exist.	Medium
BB-18	**Backup page** (1 TC)	Navigate to /backup.	URL contains /backup.	Medium
BB-19	**AI Insights** (1 TC)	Navigate to /ai.	URL contains /ai.	Medium
BB-20	**Form validation** (3 TCs)	TC01 empty fields stay on /login or show error. TC02 single-char credentials rejected. TC03 SQL injection string rejected; not logged in.	All three invalid inputs cleanly rejected.	High
BB-21	**Accessibility** (3 TCs)	TC01 skip-to-content link. TC02 all images have alt text. TC03 at least one button on /containers.	Skip link found (or skipped); no image missing alt; button count > 0.	Medium

Table 7: Black Box Test Cases — Selenium (BB-01 to BB-21)

BB Summary: 21 test groups, **48 test methods**.

Combined Selenium Suite Total: 32 test classes, 110 test methods.

9 Requirements Traceability Matrix

Requireme	Description	Test Cases
FR-01	Container discovery & metric polling	UT-01, UT-02, IT-01, ST-01, ST-06, WB-03, BB-02, BB-10
FR-02	Failure detection within 30 s	UT-03, IT-05, ST-02, BB-06
FR-03	Automated recovery actions	UT-08, UT-09, IT-06, ST-03, WB-03
FR-04	Historical metric storage via REST API	IT-02, IT-03, IT-07, ST-04, WB-03
FR-05	Real-time dashboard	IT-04, IT-08, ST-01, ST-02, BB-02
FR-06	YAML configuration and JSON export	UT-04, UT-05, UT-10, IT-09, ST-05
NFR-01	API response time $\leq$ 200 ms	PT-01
NFR-02	Host CPU overhead $<$ 5%	PT-02
NFR-03	99.9% uptime	PT-03
NFR-04	JWT authentication and TLS	UT-06, UT-07, SEC-01–04, WB-01, WB-02, WB-10, BB-01, BB-20
NFR-05	50 concurrent containers	ST-06, PT-02
WB — Auth & Security		WB-01 (password auth), WB-02 (JWT), WB-10 (access control)
WB — Container Lifecycle		WB-03 (actions), WB-04 (not found)
WB — System Health		WB-05 (health check), WB-08 (backup), WB-09 (audit)
WB — Validation & Config		WB-06 (input validation), WB-07 (thresholds), WB-11 (schema)
BB — Login & Auth flow		BB-01, BB-20
BB — Core UI pages		BB-02, BB-03, BB-04, BB-05, BB-06
BB — Feature pages		BB-10, BB-11, BB-12, BB-15–19
BB — Non-functional (UI)		BB-07, BB-08, BB-09, BB-13, BB-14, BB-21

Table 8: Requirements Traceability Matrix

10 Test Schedule and Responsibilities

Phase	Start	End	Owner
Unit Testing	Week 1	Week 2	Developers
Integration Testing	Week 3	Week 4	QA Engineers
White Box Selenium (WB-01–11)	Week 3	Week 4	Developers
System Testing	Week 5	Week 6	QA Engineers
Black Box Selenium (BB-01–21)	Week 5	Week 6	QA Engineers
Performance Testing	Week 6	Week 7	QA Engineers
Security Testing	Week 7	Week 8	Security Lead
Defect Fix & Retest	Week 8	Week 9	Developers + QA

11 Defect Management

Defects found during testing will be classified by severity:

- **Critical:** System crash, data loss, security breach — must be fixed before release.
- **High:** Core feature broken, requirement not met — must be fixed before system test sign-off.
- **Medium/Low:** Minor UI issues, non-blocking errors — tracked and fixed in next iteration.

All defects will be logged in the project issue tracker (GitHub Issues) with steps to reproduce, severity, and assigned owner.

12 Sign-Off

Role	Name	Date
Prepared By	Systems Engineering Team	May 2026
Reviewed By	QA Lead	May 2026
Approved By	Project Sponsor	May 2026